Setting up Flutter and Dart

Installation guide: Flutter SDK, VS Code, Android Studio and Xcode (Windows, Linux, macOS) and running SimpleStudentMobile

Module: Heterogeneous Systems
Group: PSe-5
Autumn 2026
Lecturer: Tadas Bulis
Kauno kolegija, Faculty of Technologies

Contents

1. Purpose
2. Requirements and preparation
3. Installation overview
4. Step 1 – prerequisites
5. Step 2 – Flutter SDK (Dart included)
6. Step 3 – configure VS Code
7. Step 4 – Android Studio and the Android SDK
8. Step 5 – Android emulator and real devices
9. Step 6 – iOS (macOS only)
10. Step 7 – check with flutter doctor
11. Running SimpleStudentMobile
12. Building the app for phones and tablets
13. Troubleshooting
14. Command cheat sheet
15. Sources

1. Purpose

This guide helps you prepare your computer for mobile development with **Flutter** (programming language **Dart**) and run the course example **SimpleStudentMobile** – a mobile client of SimpleStudentApi for Android and iOS phones and tablets.

Flutter is Google's framework that builds apps for Android, iOS, the web and desktop from one Dart code base. In our heterogeneous system it is one more technology that uses the same REST API contract (swagger.json) as the .NET and PHP clients.

1.1 What you can build on which OS

Your computer	Android app	iOS app	Note
Windows 10/11 (64-bit)	Yes	No	Apple requires macOS and Xcode to build iOS apps
Linux (64-bit)	Yes	No	This guide uses Ubuntu / Debian
macOS	Yes	Yes	Apple Silicon (M1–M4) recommended; Intel Mac support is being phased out

> **Versions (October 2026):** Flutter 3.47 (stable channel, Dart is included in the Flutter SDK), latest Android Studio, Android SDK API 36, latest Xcode. Always install the latest stable version – the commands and menus stay the same.

2. Requirements and preparation

What	Minimum	Recommended
RAM	8 GB	16 GB (the Android emulator and the IDE use a lot of memory)
Free disk space	~20 GB (Flutter + Android SDK + emulator)	40 GB+; macOS with Xcode and simulators needs more
CPU	64-bit, with virtualization (Intel VT-x / AMD-V)	Virtualization enabled in BIOS/UEFI – needed by the emulator
Permissions	Administrator rights to install software	–
Internet	Required (several GB are downloaded)	Install at home or on the college network, not on mobile data

> **Important – paths without spaces and special characters.** Flutter, Gradle and the Android tools often fail when a path contains spaces or non-ASCII letters (e.g. `C:\Users\José María\...`).
> Windows: put the Flutter SDK in `C:\dev\flutter` and your projects in `C:\dev\projects`.
> macOS / Linux: `~/develop/flutter` and `~/develop/projects`.

3. Installation overview

Step	Windows	Linux	macOS
1. Prerequisites (Git, VS Code)	Yes	Yes	Yes
2. Flutter SDK (with Dart)	Yes	Yes	Yes
3. Configure VS Code	Yes	Yes	Yes
4. Android Studio and Android SDK	Yes	Yes	Yes
5. Android emulator / real phone	Yes	Yes	Yes
6. Xcode, iOS Simulator, iPhone/iPad	–	–	Yes
7. Check: flutter doctor	Yes	Yes	Yes
8. Run SimpleStudentMobile	Yes	Yes	Yes

One editor is enough: **VS Code** (lighter, recommended for writing code) or **Android Studio**. You need Android Studio in any case – the Android SDK and the emulator are installed from it.

4. Step 1 – prerequisites

Windows

1. Download and install **Git for Windows**: https://git-scm.com/downloads/win (the default options are fine).
2. Download and install **Visual Studio Code**: https://code.visualstudio.com. During setup tick “Add to PATH” and “Open with Code”.
3. Open a new **PowerShell** window and check:

```
git --version
code --version
```

macOS

1. Install the Xcode command-line tools (they include Git) in Terminal:

```
xcode-select --install
```

2. Download **VS Code** (https://code.visualstudio.com) and move it to **Applications**.
3. In VS Code open the Command Palette (`Cmd+Shift+P`) → **Shell Command: Install 'code' command in PATH**.

Linux (Ubuntu / Debian)

```
sudo apt-get update -y && sudo apt-get upgrade -y
sudo apt-get install -y curl git unzip xz-utils zip libglu1-mesa

# VS Code (one of the options):
sudo snap install code --classic
```

On other distributions install the same packages with your package manager (dnf, pacman, etc.).

5. Step 2 – Flutter SDK (Dart included)

You do **not** need to install Dart separately – it ships inside the Flutter SDK (`flutter/bin/dart`). Choose one of the two options.

5.1 Option A – with VS Code (recommended)

1. VS Code → Extensions (`Ctrl+Shift+X`, macOS `Cmd+Shift+X`) → search **Flutter** (publisher Dart Code) → **Install**. The **Dart** extension is installed too.
2. Command Palette (`Ctrl+Shift+P` / `Cmd+Shift+P`) → type `flutter` → **Flutter: New Project**.
3. When VS Code asks for the Flutter SDK location, click **Download SDK**.
4. Choose a folder (Windows `C:\dev`, macOS/Linux `~/develop`) → **Clone Flutter**. The download takes a few minutes.
5. When offered, click **Add SDK to PATH**.
6. Close all terminals and **restart VS Code**. You can cancel the new-project dialog.

5.2 Option B – manual installation

Download the latest **stable** bundle: https://docs.flutter.dev/install/archive (macOS – choose Apple Silicon or Intel).

Windows (PowerShell)

```
New-Item -ItemType Directory -Force C:\dev
Expand-Archive -Path "$env:USERPROFILE\Downloads\flutter_windows_3.47.x-stable.zip" -
DestinationPath C:\dev
# Result: C:\dev\flutter\bin\flutter.bat
```

PATH: `Win + Pause` (or Start → “Edit the system environment variables”) → **Advanced system settings** → **Environment Variables…** → **User variables** → **Path** → **Edit** → **New** → `C:\dev\flutter\bin` → **Move Up** to the top → **OK** ×3. Open a new terminal.

> If `flutter.bat` is missing from the `bin` folder after extracting, your antivirus has probably quarantined it – add `C:\dev\flutter` to the exclusions and extract again.

macOS (zsh)

```
mkdir -p ~/develop
unzip ~/Downloads/flutter_macos_arm64_3.47.x-stable.zip -d ~/develop/
echo 'export PATH="$HOME/develop/flutter/bin:$PATH"' >> ~/.zprofile
source ~/.zprofile
```

Linux (bash)

```
mkdir -p ~/develop
tar -xf ~/Downloads/flutter_linux_3.47.x-stable.tar.xz -C ~/develop/
echo 'export PATH="$HOME/develop/flutter/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
# zsh users: >> ~/.zshenv
```

5.3 Check

```
flutter --version
dart --version
flutter doctor
```

The first `flutter` run downloads extra tools – that is normal. At this point `flutter doctor` still reports missing Android / Xcode – we install them next. To update later: `flutter upgrade`.

6. Step 3 – configure VS Code

If you used option B, install the **Flutter** extension (Dart Code) in VS Code now – see step 1 of 5.1. The extension finds the Flutter SDK through PATH.

6.1 Useful commands and shortcuts

Action	Windows / Linux	macOS
Command Palette	`Ctrl+Shift+P`	`Cmd+Shift+P`
Flutter: Run Flutter Doctor	Command Palette	Command Palette
Select device (emulator, phone)	Status bar, bottom right, or **Flutter: Select Device**	same
Run with debugging	`F5`	`F5`
Run without debugging	`Ctrl+F5`	`Ctrl+F5`
Hot reload (changes without restart)	Save the file `Ctrl+S`	`Cmd+S`
Hot restart (state is reset)	`Ctrl+Shift+F5`	`Cmd+Shift+F5`
Get packages (pub get)	Automatic when pubspec.yaml changes, or **Flutter: Get Packages**	same
Flutter DevTools	**Flutter: Open DevTools**	same

6.2 Recommended settings

Command Palette → **Preferences: Open User Settings (JSON)** and add:

```
"[dart]": {
  "editor.formatOnSave": true,
  "editor.rulers": [80]
},
"dart.previewFlutterUiGuides": true
```

Open a project with **File → Open Folder…** and select the project folder itself (the one with `pubspec.yaml`), not the whole repository.

7. Step 4 – Android Studio and the Android SDK

7.1 Installing Android Studio

Download the latest stable version: https://developer.android.com/studio

OS	How to install
Windows	Run the `.exe` installer, keep “Android Virtual Device” ticked.
macOS	Open the `.dmg` and drag **Android Studio** to **Applications**. On Apple Silicon choose the “Mac with Apple chip” download.
Linux	Extract the `.tar.gz` (e.g. to `/opt/android-studio`) and run `bin/studio.sh`, or: `sudo snap install android-studio --classic`. First install the 32-bit libraries needed on 64-bit systems – list: https://developer.android.com/studio/install#64bit-libs

In the first-run **Setup Wizard** choose **Standard** and let it download the suggested components.

7.2 SDK Manager

Welcome screen **More Actions** → **SDK Manager** (with a project open: **Tools** → **SDK Manager**).

- **SDK Platforms** tab: tick **Android API 36** (the first entry with API Level 36).
- **SDK Tools** tab: **Android SDK Build-Tools**, **Android SDK Command-line Tools**, **Android Emulator**, **Android SDK Platform-Tools**, **CMake**, **NDK (Side by side)**.
- Click **Apply** → **OK** → **Finish**.

> **Android SDK Command-line Tools** is often forgotten. Without it `flutter doctor` reports “cmdline-tools component is missing” and you cannot accept the licenses.

7.3 Flutter plugin for Android Studio

1. Windows/Linux: **File** → **Settings** → **Plugins**; macOS: **Android Studio** → **Settings** → **Plugins**.
2. In **Marketplace** search **Flutter** → **Install** (it also offers **Dart**) → **Restart IDE**.
3. After the restart the Welcome screen shows **New Flutter Project**. If asked for the Flutter SDK path, enter `C:\dev\flutter` or `~/develop/flutter`.

7.4 Android licenses

```
flutter doctor --android-licenses
# answer y to every license
```

If Flutter cannot find the Android SDK, set the path: `flutter config --android-sdk <path>`. Default paths: Windows `%LOCALAPPDATA%\Android\Sdk`, macOS `~/Library/Android/sdk`, Linux `~/Android/Sdk`. You do not need to install Java (JDK) separately – Flutter uses the JDK bundled with Android Studio.

8. Step 5 – Android emulator and real devices

8.1 Virtualization (emulator speed)

OS	What to do
Windows	Enable **Intel VT-x** or **AMD-V / SVM** in BIOS/UEFI. Then Start → `optionalfeatures` → tick **Windows Hypervisor Platform** → restart the computer.
Linux	KVM is needed: `sudo apt install -y qemu-kvm` and `sudo adduser $USER kvm`, then log out and log in again. Check: `ls -l /dev/kvm`.
macOS	Nothing to do. On Apple Silicon use **ARM** system images (Android Studio offers them automatically).

8.2 Creating emulators (phone and tablet)

1. Android Studio: **More Actions** → **Virtual Device Manager** (or **Tools** → **Device Manager**) → **+ / Create Virtual Device**.
2. In the **Phone** category choose e.g. **Pixel 9** → **Next**.
3. Choose a system image with **API 36** (x86_64 for Intel/AMD computers, ARM for Apple Silicon) → download it → **Next**.
4. **Additional settings** → **Emulated Performance** → **Graphics acceleration**: choose an option with **Hardware** → **Finish**.
5. Repeat with the **Tablet** category (e.g. **Pixel Tablet**) to see how the app looks on a tablet.
6. Start the emulator with the ▶️ button in Device Manager.

```
flutter emulators                # list emulators
flutter emulators --launch <id>  # start one
flutter devices                  # the device must be listed
```

8.3 Real Android phone or tablet

1. On the phone: **Settings** → **About phone** → tap **Build number** 7 times – **Developer options** are enabled.
2. **Settings** → **System** → **Developer options** → turn on **USB debugging** (optionally **Wireless debugging**).
3. **Windows:** install the manufacturer's USB driver (https://developer.android.com/studio/run/oem-usb). macOS and Linux need no drivers.
4. Connect the USB cable and confirm “Allow USB debugging?” on the phone.
5. Check: `flutter devices`.

9. Step 6 – iOS (macOS only)

9.1 Xcode

1. Install the latest **Xcode** from the Mac App Store or https://developer.apple.com/xcode/ (large download, takes a while).
2. In Terminal configure the tools, accept the license and download the iOS platform with simulators:

```
sudo sh -c 'xcode-select -s /Applications/Xcode.app/Contents/Developer && xcodebuild -
runFirstLaunch'
sudo xcodebuild -license
xcodebuild -downloadPlatform iOS
```

9.2 CocoaPods and Swift Package Manager

Flutter 3.47 uses **Swift Package Manager** for iOS dependencies by default (nothing to install). Still, `flutter doctor` reports **"× CocoaPods not installed"**, because CocoaPods is needed for plugins that do not support SPM yet. Install it – it takes a few minutes.

1) Homebrew (if you do not have it yet)

Check: `brew --version`. If the command is not found, install Homebrew (https://brew.sh) – paste into Terminal:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

The installer asks for your Mac password (it is not shown while typing – that is normal) and for **Enter** to confirm. On **Apple Silicon (M1–M4)** it prints "Next steps" at the end – run them so that `brew` is on your PATH:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
brew --version          # must print a version
```

On an Intel Mac Homebrew installs into `/usr/local` and no PATH change is needed.

2) CocoaPods

```
brew install cocoapods
pod --version           # e.g. 1.16.x
flutter doctor          # the Xcode line must show [✓]
```

Nothing else to do: Flutter runs `pod install` itself during `flutter run` / `flutter build ios` when the project has plugins that use CocoaPods.

> The recommended way is **Homebrew**. `sudo gem install cocoapods` with the macOS system Ruby often fails (Ruby too old, permission problems), so do not use it unless you know what you are doing (e.g. you use rbenv / rvm).

Common errors

Error	Solution
`brew: command not found` (after installing Homebrew)	The "Next steps" were not run – run the two `brew shellenv` lines from step 1) and open a new terminal.
`pod: command not found` or doctor still shows ×	Close the terminal and VS Code / Android Studio and open them again. Check `which pod` (should be `/opt/homebrew/bin/pod`).
"CocoaPods installed but not working"	`brew reinstall cocoapods`. If it was installed with gem before: `sudo gem uninstall cocoapods`, then `brew link --overwrite cocoapods`.
"CocoaPods's specs repository is too out-of-date"	`pod repo update`, then `flutter clean` and `flutter run`.
Errors during `pod install` in a project	`cd ios && rm -rf Pods Podfile.lock && cd ..`, then `flutter clean`, `flutter pub get`, `flutter run`.

9.3 iOS Simulator (iPhone and iPad)

```
open -a Simulator          # Xcode 26 and earlier
open -a DeviceHub          # Xcode 27 (Simulator was renamed)
flutter emulators --launch apple_ios_simulator
```

Choose another device (e.g. an **iPad**) in the simulator menu **File → Open Simulator**. iPad is supported automatically – the Flutter iOS project is created with `TARGETED_DEVICE_FAMILY = "1,2"` (iPhone + iPad).

9.4 Real iPhone or iPad

1. Connect with a USB cable and tap **Trust** on the device.
2. On the device: **Settings → Privacy & Security → Developer Mode → On** → restart → **Turn On**. If the option is missing, first run the app from Xcode or with `flutter run`.
3. Open `ios/Runner.xcworkspace` in Xcode → **Runner → Signing & Capabilities → Team**: choose your Apple ID (a free account is enough: **Xcode → Settings → Accounts → +**).
4. Change the **Bundle Identifier** to a unique one, e.g. `lt.kaunokolegija.simpleStudentMobile.yourname`.
5. First time on the device: **Settings → General → VPN & Device Management** → your certificate → **Trust**.

> With a free Apple account the installed app works for **7 days**; then run it again from Xcode / `flutter run`. Publishing to the App Store requires the paid Apple Developer Program.

10. Step 7 – check with flutter doctor

```
flutter doctor

Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.x, ...)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.x)
[✓] Xcode - develop for iOS and macOS (Xcode 2x.x)        <- macOS only
[✓] Chrome - develop for the web
[✓] Android Studio (version 202x.x)
[✓] VS Code (version 1.1xx)
[✓] Connected device (2 available)
[✓] Network resources

• No issues found!
```

Line	Required for this course?
Flutter, Android toolchain, Android Studio	**Yes** – on every OS
Xcode	**Yes** on macOS only, if you build the iOS version
VS Code	If you use it
Chrome	No (only for the web) – can be ignored
Visual Studio (Windows)	No – only for Windows desktop apps. Do not install it.
Linux toolchain (Linux)	No – only for Linux desktop apps

More details and suggested fixes: `flutter doctor -v`. All Android/Xcode items must show [✓] before you continue.

11. Running SimpleStudentMobile

The project is in `code-examples/student-docker-lab-plus-mssql/SimpleStudentMobile` (in the course GitHub repository – in the `student-docker-lab-plus-mssql` folder). It has two tabs: **Home** (project info, API connection test) and **Students** (CRUD: list, create, edit, delete).

11.1 Start the API

```
cd student-docker-lab-plus-mssql
docker-compose -f docker-compose.infra.yml up -d   # SQL Server (once)
docker-compose up --build -d                        # .NET API :6001, PHP API :6002
# check in the browser: http://localhost:6001/swagger
```

XAMPP users: instead of Docker you can use the PHP API from `SimpleStudentPhpApi` (`htdocs/simpleStudentApi`, MySQL) – see the address in table 11.4.

11.2 Prepare the Flutter project

```
cd SimpleStudentMobile
flutter pub get
```

The project already contains prepared `android/` and `ios/` folders that allow the app to call the API over **HTTP**:

- `android/app/src/main/AndroidManifest.xml` – `INTERNET` permission and `android:usesCleartextTraffic="true"`;

- `ios/Runner/Info.plist` – `NSAppTransportSecurity` (`NSAllowsArbitraryLoads`) and `NSLocalNetworkUsageDescription`.

> **If the `android/` or `ios/` folder is missing** (deleted, or you copied only `lib/`) or the HTTP permissions are missing, run `dart run tool/configure_platforms.dart`. It generates the missing folders with `flutter create` (existing files are not overwritten) and adds the HTTP permissions. Safe to run any number of times.

> Android 9+ and iOS block unencrypted HTTP by default. We allow it only because this is a lab – a real app uses HTTPS.

11.3 Run the app

- **VS Code:** File → Open Folder → `SimpleStudentMobile` → choose the device at the bottom → `F5`.
- **Android Studio:** File → Open → `SimpleStudentMobile` → choose the emulator in the device list at the top → ▶️ **Run**.
- **Terminal:** `flutter run` (several devices: `flutter run -d <id>`).

11.4 API address

The phone is **another computer**, so `localhost` does not always mean your PC. The app picks a default address itself; change it on the **Home** tab → **Change** (saved on the device).

Where the app runs	API address
Android emulator	`http://10.0.2.2:6001` (10.0.2.2 = your computer)
iOS Simulator (Mac)	`http://localhost:6001`
Real Android via USB	`http://localhost:6001` + `adb reverse tcp:6001 tcp:6001` (see 11.6)
Real phone / tablet via Wi-Fi	`http://<computer IP>:6001` – on the same Wi-Fi network (see 11.6)
PHP API in Docker	same, but port `6002`
PHP API in XAMPP	`http://10.0.2.2/simpleStudentApi` (emulator), `http://localhost/simpleStudentApi` (iOS sim.); if XAMPP Apache runs on 8080 – add `:8080`

You can also set the address at start-up: `flutter run --dart-define=API_BASE_URL=http://192.168.1.20:6001`.

Your computer's IP: Windows `ipconfig` (IPv4 Address), macOS `ipconfig getifaddr en0`, Linux `hostname -I`.

Firewall (real devices only)

```
# Windows (PowerShell as Administrator):
New-NetFirewallRule -DisplayName "SimpleStudentApi 6001" -Direction Inbound -Protocol TCP -LocalPort 6001 -Action Allow

# Linux (ufw):
sudo ufw allow 6001/tcp

# macOS: System Settings -> Network -> Firewall (allow Docker / com.docker.backend)
```

11.5 What to try

1. **Home** → **Test connection**: shows “SimpleStudentApi v1.2 – ok” (data from `GET /api/info`).
2. **Students**: the list (`GET /api/students`); pull down to refresh.

3. **+ Add** → fill in the form → **Create student** (`POST`). Enter an invalid email – the error appears under the field.
4. Tap a student → change something → **Save changes** (`PUT`). The bin icon deletes (`DELETE`).
5. Swipe a student to the left in the list – delete with confirmation.
6. Rotate the emulator to landscape or run on a tablet – the bottom menu becomes a side menu (**NavigationRail**).
7. Open the same list in the browser (http://localhost:6011 or :6021) – the same data: three clients, one API.
8. Change a text in `lib/screens/home_screen.dart` and save – **hot reload** updates the screen within a second.

11.6 The app cannot reach the API (Test connection is red)

The most common cause is an API address that does not fit **your environment**. A phone or tablet is a separate computer, so `localhost` there means the phone itself, and `10.0.2.2` works **only in the Android emulator** (it is the emulator's alias for the computer it runs on). On a real device and in the computer's browser `10.0.2.2` leads nowhere.

> Typical error on a real device: “The API did not answer within 10 s … Is the address http://10.0.2.2:6001 correct?” – the app is using the emulator address. Pick an address from the table below.

Which address to use

Your environment	Address in the app (Home → Change)	What to do
Android emulator	`http://10.0.2.2:6001`	Nothing – this is the default
iOS Simulator (Mac)	`http://localhost:6001`	Nothing – this is the default
Real Android connected by **USB** (recommended in class)	`http://localhost:6001`	`adb reverse tcp:6001 tcp:6001` (see option A)
Real Android or iPhone/iPad over **Wi-Fi**	`http://<computer IP>:6001`	Same Wi-Fi network, allowed in the firewall (see option B)

Option A – USB and adb reverse (Android only)

`adb reverse` forwards the phone's `localhost:6001` through the USB cable to port 6001 on your computer. Wi-Fi and the firewall do not matter, so this is the most reliable option on the college network.

```
adb devices                          # the phone must be listed with state "device"
adb reverse tcp:6001 tcp:6001        # answers: 6001
adb reverse --list                   # check

# several devices connected (e.g. an emulator is running too):
adb -s <device ID> reverse tcp:6001 tcp:6001
# the device ID is shown by: flutter devices  (e.g. R9TX60KABKK)
```

Then in the app **Home** → **Change** → `http://localhost:6001` → **Test connection**. Or set it at start-up:

```
flutter run -d <device ID> --dart-define=API_BASE_URL=http://localhost:6001
```

If the `adb` command is not found, use the full path:

OS	Path to adb
Windows	`%LOCALAPPDATA%\Android\Sdk\platform-tools\adb.exe`

OS	Path to adb
macOS	`~/Library/Android/sdk/platform-tools/adb`
Linux	`~/Android/Sdk/platform-tools/adb`

> `adb reverse` lasts while the device stays connected. After unplugging the cable or restarting the phone or adb, run the command again. iPhone/iPad have no equivalent – use option B for them.

Option B – Wi-Fi and the computer's IP address

1. Find your computer's IP address on the Wi-Fi network: Windows `ipconfig` (line **IPv4 Address** of the Wi-Fi adapter), macOS `ipconfig getifaddr en0`, Linux `hostname -I` (first address). E.g. `192.168.1.29`.
2. Open `http://192.168.1.29:6001/swagger` in the phone's **browser**. If it opens, the network is fine.
3. In the app **Home** → **Change** → `http://192.168.1.29:6001` → **Test connection**.

If the page does not open in step 2, the problem is the network, not the app:

- The **firewall** blocks port 6001 – allow it (commands in 11.4; macOS: System Settings → Network → Firewall → Options → allow Docker / `com.docker.backend`).
- The **Wi-Fi isolates devices** (common on guest and public networks, sometimes on the college network) – use option A (USB) or the phone's hotspot and connect the computer to it.
- **Different network** – the phone is on mobile data or another Wi-Fi.

> The computer's IP can change when it reconnects to Wi-Fi – if the connection stops working later, check the IP again.

Checklist (any environment)

1. Does `http://localhost:6001/swagger` open in the computer's browser? If not, the API is not running: `docker ps`, `docker-compose up --build -d` (11.1).
2. Does the address set in the app fit your environment (table above)?
3. On a real device: does the same address + `/swagger` open in the device's browser?
4. Was the address changed with **Change**? It takes priority over `--dart-define`. To go back to the default: **Change** → **Use default**.
5. Error “Cleartext HTTP traffic not permitted” – run `dart run tool/configure_platforms.dart` and restart the app (not hot reload).

12. Building the app for phones and tablets

Target	Command	Result
Android APK (install on any phone / tablet)	`flutter build apk --release`	`build/app/outputs/flutter-apk/app-release.apk`
Android App Bundle (Google Play)	`flutter build appbundle`	`build/app/outputs/bundle/release/app-release.aab`
Install on a connected device	`flutter install`	Installs the last built APK
iOS (macOS + Xcode only)	`flutter build ios --release`	Then in Xcode: **Product → Archive**
Install on iPhone/iPad so it opens from the home screen	`flutter run --release -d <ID>`	The release build stays on the device; `q` in the terminal to stop

For the lab an APK is enough – you can send it and install it on a phone (allow “Install unknown apps”). The release APK is signed with the debug key, so it is not suitable for Google Play; that needs a separate signing key.

12.1 iOS: the app does not open from the home screen

When you open a **debug** build from the home screen, the iPhone shows: “In iOS 14+, debug mode Flutter apps can only be launched from Flutter tooling, IDEs with Flutter plugins or from Xcode.” This is an iOS restriction: a debug build (JIT, hot reload) only runs while a debugger is attached. No project setting can change that.

The fix is to install a **release** (or profile) build. It opens like a normal app:

```
flutter devices
flutter run --release -d <iPhone ID> --dart-define=API_BASE_URL=http://<computer IP>:6001
```

- After it is installed you can press `q` in the terminal – the app stays on the iPhone and opens from the home screen.
- Release builds have no hot reload – for development keep using the normal `flutter run` (debug).
- With a free Apple account the app works for 7 days – then install it again with the same command.
- From Xcode: **Product → Scheme → Edit Scheme… → Run → Build Configuration → Release**, then **Run**.
- Android has no such restriction – a debug APK opens from the home screen.

13. Troubleshooting

Problem	Solution
`flutter` / `dart` is not recognized / command not found	PATH is not set or the terminal is old. Check PATH, close ALL terminals and VS Code, open them again.
`cmdline-tools component is missing`	SDK Manager → SDK Tools → tick **Android SDK Command-line Tools** → Apply.
`Android license status unknown`	`flutter doctor --android-licenses`, answer `y`.
`Unable to locate Android SDK`	`flutter config --android-sdk <path to Sdk>`
Gradle / Java version errors	Update Android Studio. If needed: `flutter config --jdk-dir <Android Studio>/jbr` (macOS: `.../Android Studio.app/Contents/jbr/Contents/Home`).

Problem	Solution
Gradle errors with strange characters in the path	Move the project and the SDK to a path without spaces and non-ASCII letters (`C:\dev\...`).
The emulator is very slow / does not start	Enable virtualization (8.1), Graphics – Hardware, close other heavy apps. Linux – check permissions of `/dev/kvm`.
`Cannot connect to http://...`	Is the API running (http://localhost:6001/swagger)? Correct address for your device (11.4)? Firewall? See **11.6**.
On a real device: "... Is the address http://10.0.2.2:6001 correct?"	`10.0.2.2` works only in the emulator. USB: `adb reverse tcp:6001 tcp:6001` and address `http://localhost:6001`; Wi-Fi: `http://<computer IP>:6001` (see 11.6).
`Cleartext HTTP traffic not permitted` (Android)	Run `dart run tool/configure_platforms.dart` and restart the app fully (not hot reload).
`App Transport Security has blocked a cleartext HTTP` (iOS)	Same script; check `ios/Runner/Info.plist`.
iPhone: "In iOS 14+, debug mode Flutter apps can only be launched…"	Install a release build: `flutter run --release -d <ID>` (see 12.1).
Xcode: `could not find included file 'Generated.xcconfig'`	Run `flutter pub get` in the project folder first, then open `ios/Runner.xcworkspace`.
`CocoaPods not installed` (macOS)	Install it as in 9.2 (Homebrew → `brew install cocoapods`), then `flutter doctor`.
iOS: "No valid code signing certificates"	Xcode → Runner → Signing & Capabilities → choose a Team, change the Bundle Identifier.
Strange package / build errors after an update	`flutter clean`, then `flutter pub get` and run again.
Windows: `flutter.bat` disappeared	Antivirus quarantine – add `C:\dev\flutter` to the exclusions and extract again.

14. Command cheat sheet

Command	Purpose
`flutter doctor` / `flutter doctor -v`	Check the environment
`flutter upgrade`	Update Flutter and Dart
`flutter create <name>`	New project (name in lowercase with _)
`flutter pub get` / `flutter pub add http`	Get packages / add a package
`flutter devices` / `flutter emulators`	Connected devices / emulators
`flutter run` / `flutter run -d <id>`	Run (in the terminal: `r` – hot reload, `R` – hot restart, `q` – quit)
`flutter analyze`	Static code analysis
`flutter test`	Tests
`dart format .`	Format the code
`flutter clean`	Clean the build folder
`flutter build apk` / `flutter build ios`	Build the app

15. Sources

- Install Flutter with VS Code: https://docs.flutter.dev/install/quick
- Install Flutter manually: https://docs.flutter.dev/install/manual
- Android setup for Flutter: https://docs.flutter.dev/platform-integration/android/setup
- iOS setup for Flutter: https://docs.flutter.dev/platform-integration/ios/setup
- Flutter SDK archive: https://docs.flutter.dev/install/archive
- Android Studio: https://developer.android.com/studio
- Dart language: https://dart.dev/language
- http package: https://pub.dev/packages/http